

Lab 0: Warmup — Environment & C Refresher

CSCI 3334 · UTRGV Fall 2026

Overview

Warmup — worth up to 3 bonus points toward your final grade. This lab verifies that your development environment works and you remember enough C to start the course. It's pure upside: finishing it adds points, and there's no penalty for anything you can't complete. If you struggle with any part, that's useful information — not a problem. Review the relevant sections and come to office hours.

Due: Wed Sep 9 (session 4)

Objectives

1. Set up Git, GCC, Make, GDB, Valgrind
2. Write and compile a C program
3. Debug with GDB
4. Detect memory errors with Valgrind

Part 1: Environment Setup

Linux (recommended)

```
sudo apt update
sudo apt install build-essential gdb valgrind git
```

macOS

```
xcode-select --install
brew install gdb valgrind # Valgrind may not work on Apple Silicon
```

Windows — WSL2

Native Windows is not supported. Use **WSL2** (Windows Subsystem for Linux), which runs a real Ubuntu environment inside Windows 10 (version 2004+) or Windows 11.

1. Install WSL2. Open **PowerShell** or **Windows Terminal** as Administrator (right-click → “Run as administrator”) and run:

```
wsl --install
```

This enables the required Windows features and installs Ubuntu by default. Reboot when prompted.

2. Create your Linux user. After the reboot, an Ubuntu terminal opens and asks you to pick a UNIX username and password. (This is separate from your Windows login; the password is what `sudo` will ask for — typing shows nothing, that's normal.)

3. Install the toolchain. Inside the Ubuntu terminal:

```
sudo apt update
sudo apt install build-essential gdb valgrind git
```

4. Work inside the Linux filesystem. Keep your lab files under your Linux home (~, i.e. /home/<you>), **not** under /mnt/c/.... Building on /mnt/c is slow and can confuse Git. From the Ubuntu terminal, `cd ~` and clone there.

Tips

- Reopen the environment any time by launching **Ubuntu** from the Start menu, or run `wsl` from PowerShell.
- If `wsl --install` reports an unrecognized command, your Windows is too old — update Windows, or follow Microsoft's manual install steps at <https://learn.microsoft.com/windows/wsl/install>.
- To edit files with VS Code, install the **WSL** extension, then run `code .` from inside the Ubuntu terminal.

Windows alternative

If WSL2 will not run on your machine (e.g. an older Windows build or a locked-down device), a Linux VM (VirtualBox or VMware with Ubuntu) works too.

Verify

```
gcc --version          # Should be >= 11
gdb --version
valgrind --version
git --version
```

Part 2: Hello, GDB

Clone the starter repo:

```
git clone https://github.com/jpcoding/csci3334-labs.git
cd csci3334-labs/lab0
make
./hello
```

You should see: Hello, Systems!

Now debug it:

```
gdb ./hello
(gdb) break main
(gdb) run
(gdb) step
(gdb) print <variable>
(gdb) continue
(gdb) quit
```

Submit: A screenshot showing GDB stopped at a breakpoint inside `main`.

Part 3: Fix the Memory Bug

`leaky.c` contains a deliberate memory bug. Compile and run:

```
make leaky
./leaky
```

It prints numbers. But it leaks memory. Run Valgrind:

```
valgrind --leak-check=full ./leaky
```

Valgrind will report something like:

==12345== LEAK SUMMARY:

==12345== definitely lost: XX bytes in Y blocks

Your task: Find and fix the leak. Run Valgrind again to confirm zero leaks.

Submit: Your fixed `leaky.c`

Part 4: Write a Linked List

Write `l1list.c` that implements a singly-linked list of integers:

```
typedef struct node {
    int value;
    struct node *next;
} Node;

// Create a new node
Node *node_new(int value);

// Append to end of list
void list_append(Node **head, int value);

// Print all values
void list_print(Node *head);

// Free entire list
void list_free(Node *head);
```

Write a `main` in `l1list.c` that:

1. Creates a list with values [1, 2, 3, 4, 5]
2. Prints it
3. Frees all memory

Run Valgrind: zero leaks, zero errors.

Submit: `l1list.c`

Grading (up to 3 bonus points)

One point per part — all three verify only that your environment works and your code runs cleanly, not how elegant it is.

Requirement	Points
Part 2: GDB screenshot stopped at a breakpoint in <code>main</code>	1
Part 3: Fixed <code>leaky.c</code> , Valgrind reports zero leaks	1
Part 4: <code>l1list.c</code> compiles, works, Valgrind clean	1

Points are bonus — added to your final grade, never subtracted. Anything you can't finish is a signal to come to office hours, not a penalty.

Hints

- `sizeof(Node)` does not include the data pointed to by `next` — you must free each node individually
- In GDB, `p/x var` prints a variable in hex
- Valgrind's "definitely lost" = serious; "still reachable" = usually fine (but fix it anyway)